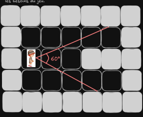
RAYCASTING WITH CUBE 3D

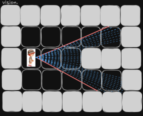
Le raycasting est une technique de rendu graphique utilisée pour créer une vue en 3D à partir d'une carte en 2D. Elle est principalement utilisée dans les jeux vidéo pour simuler un environnement 3D, notamment dans les jeux de tir à la première personne (FPS). Voici comment fonctionne le raycasting :

1. Position et Orientation du Joueur : Le point de départ est la position du joueur dans l'environnement 2D. L'orientation du joueur (la direction dans laquelle il regarde) est également prise en compte.

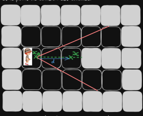
2. Champ de Vision (FOV) :
Un angle de champ de vision est défini pour déterminer la largeur de la vue du joueur. Cet angle est généralement de 60 à 90 degrés, mais peut varier selon les besoins du jeu.



3. Projection des Rayons :
Pour chaque colonne de pixels à l'écran, un rayon est projeté depuis la position du joueur à travers le champ de vision. Ces rayons sont projetés à intervalles réguliers pour couvrir tout le champ de vision.



4. Détection des Obstacles :
Chaque rayon est projeté jusqu'à ce qu'il rencontre un obstacle, comme un mur. La distance entre le joueur et le point de collision est calculée.



Les coordonees d'un point sur le rayon a une distance d du joueur s'ecrit comme ca :

x(d) = x0 + d * cos(θ)

et

y(d) = y0 + d * sin(θ)

ou (x, y) correspondent aux coordonees du point sur le rayon, (x0, y0) correspondent aux coordonees du joueur.
d = distance du rayon
θ = angle du rayon exprimer en radian !

CALCUL De la distance entre le joueur et le point de collision :

Contexte :

- Position du joueur : (x0, y0)
- Angle du rayon : θ
- Carte : grille 2d ou chaques cellules est un mur ou un espace vide
- On cherche a trouver ou le rayon touche un mur et a quelle distance

1. Equation du rayon :

La rayon est un droite paramétrée :
x(d) = x0 + d * cos(θ)
y(d) = y0 + d * sin(θ)

ou d >= 0 est une distance parcourue depuis la position du joueur.
PERMET DE SUIVRE LE RAYON EN TOUT POINTS POUR CHECK SI UN MUR EST ATTEINT.

2. Objectif du raycasting :

Trouver le plus petit d tel que le point (x(d), y(d)) tombe dans une cellule de mur de la grille.

3. Methode (DDA - Digital Differential Analyzer)

- Etape 1 : Calcul du vecteur direction

Determine la direction du rayon (haut, bas, gauche, droite :

$dx = \cos(\theta)$

$dy = \sin(\theta)$

SI : dx > 0, alors DROITE
SI : dx < 0, alors GAUCHE
SI : dy > 0, alors BAS
SI : dy < 0, alors HAUT

- Etape 2 : Calcul des distances pour traverser une cellule de la grille.

$$\Delta distX = \frac{1}{|\cos(\theta)|}$$

Correspond a la distance du rayon pour traverser une unite en X (horizontalement).

$$\Delta distY = \frac{1}{|\sin(\theta)|}$$

Correspond a la distance du rayon pour traverser une unite en Y (Verticalement).

- Etape 3 : Initialization des distance laterales :

Determiner la case ou est le joueur : (map_x, map_y)
si le joueur est a la position : (x0, y0) = (2.2, 3.4)
alors : (map_x, map_y) = (2, 3)

SideDistX : Correspond a la distance en X de la position exacte du joueur a la sortie de sa case actuelle
SideDistY : Correspond a la distance en Y de la position exacte du joueur a la sortie de sa case actuelle

SI : dx > 0

$$SideDistX = \left(1 - \left(x_0 - map_x\right)\right) \cdot \Delta distX$$

SI : dx < 0

$$SideDistX = \left(y_0 - map_x\right) \cdot \Delta distX$$

SI : dy > 0

$$SideDistY = \left(1 - \left(y_0 - map_y\right)\right) \cdot \Delta distY$$

SI : dy < 0

$$SideDistY = \left(y_0 - map_y\right) \cdot \Delta distY$$

- Etape 4 : mise a jour des distances laterales et deplacement du rayon

A chaques fois, selectionner le SideDist la plus courte :

le SideDist de X ou Y sera alors mis a jour en lui ajoutant Δdist X ou Y qui correspond je le rappel a la distance du rayon pour traverser une unite en X ou Y.
Comme la distance entre la position exacte du joueur a la sortie de sa case actuelle est deja calculer, il suffit donc simplement d'ajouter Δdist X ou Y.

SI : SideDistX < SideDistY

$$SideDistX = SideDistX + \Delta distX$$

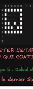
SI dx > 0 alors map_x += 1 (ON AVANCE DANS L'AXE DES ABSCISSES)
SI dx < 0 alors map_x -= 1 (ON RECULE DANS L'AXE DES ABSCISSES)

SI : SideDistY < SideDistX

$$SideDistY = SideDistY + \Delta distY$$

SI dy > 0 alors map_y += 1 (ON DESCEND DANS L'AXE DES ORDONNEES (y + 1))
SI dy < 0 alors map_x -= 1 (ON MONTE DANS L'AXE DES ORDONNEES (y - 1))

Cela peut paraitre paradoxale :
quand dy > 0, cela veut dire sur le plan mathematique que l'angle dirige vers le haut, ce qui voudrait logiquement dire que l'on se deplace vers le haut dans la map
NEANMOINS,
l'axe des ordonnee est inverser dans la representation de la map :



plus y augmente plus on descend dans les ordonnees.
l'axe des ordonnee mathematique et d'une map est inverser, donc on inverse la logique des angles.
dy > 0 = DESCEND dans les ordonnee.
dy < 0 = MONTE dans les ordonnee.
alors qu'en maths :
plus y augmente = MONTE dans les ordonnee.
plus y baisse = DESCEND dans les ordonnee.
(j'espere avoir ete clair sur cette partie !)

REPETER L'ETAPE JUSQU'A TOMBER SUR UNE CASE QUI CONTIENT UN MUR !

- Etape 5 : Calcul de la distance entre le joueur et le mur

SI : le dernier SideDist effectue est X :

$$d = \frac{\left(map_x - x_0 + \frac{(1 - stepX)}{2}\right)}{\cos(\theta)}$$

d = distance final entre joueur et mur
map_x = emplacement du mur en x
x_0 = abscisse de la position du joueur
SI dx > 0 alors stepX = 1
SI dx < 0 alors stepX = -1

SI : le dernier SideDist effectue est Y :

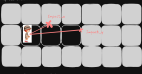
$$d = \frac{\left(map_y - y_0 + \frac{(1 - stepY)}{2}\right)}{\sin(\theta)}$$

d = distance final entre joueur et mur
map_y = emplacement du mur en y
y_0 = ordonnee de la position du joueur
SI dy > 0 alors stepY = 1
SI dy < 0 alors stepY = -1

CALCUL Du ratio WALL_HIT :

Pour la suite, il va nous falloir le ratio de la position exact de colision du rayon avec le mur, ce ratio est donc definie sur [0, 1]. 0 = le plus a gauche ou en haut 1 = le plus a droite ou en bas.
nous appelerons cette variable wall_hit.

Pour calculer wall_hit, il nous faut a ce moment, le position de l'impact exact du rayon sur le mur. Prenons l'exemple de dx > 0 soit une colision sur une face horizontal d'un mur, plus precisemment sur une face droite.
Representation :



Ce Point d'impact est calculer comme suis :

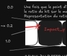
SI dx > 0 OU dx < 0 (impact d'une face verticale)

$$Impact_y = player_y + rayon_{dist} \cdot dy$$

SI dy > 0 OU dy < 0 (impact d'une face horizontale)

$$Impact_x = player_x + rayon_{dist} \cdot dx$$

Une fois que le point d'impact exact est localiser, il nous suffit de calculer le ratio du hit sur le mur (wall_hit)
Representation du ratio :



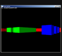
La face du mur entier fait 1.0
les deux extremes cotes de la face sont representer par 0.0 et 1.0
le wall hit est un ration normaliser qui permet de definir ou on se trouve dans la face et donc de definir apres quel colonne de la texture imprimer

CALCUL DU RATIO WALL_HIT :

$$RATIO = \frac{\left(Impact_{x\ or\ y} \mod Case_{width\ or\ height}\right)}{Case_{width\ or\ height}}$$

RATIO = Wall(hit)
Case_width if (impact_y)
Case_height if (impact_x)
mod = Operateur Modulo
Case = taille d'une case en 2D au niveau de la representation 2d du Cub3d

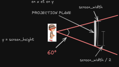
5. Calcul de la hauteur des murs :



La distance calculée est utilisée pour déterminer la hauteur des murs à l'écran. Plus l'obstacle est proche, plus le mur apparaîtra grand sur l'écran. Cela crée une illusion de profondeur et de perspective.

- Etape 1 : Calcul de la distance entre le joueur et le plan de projection

La projection des murs et autres doivent etre fait en suivant un plan de projection, le plan de projection correspond a l'ecran que le joueur verra, soit la taille de sont ecran en x et en y



Distance du joueur au projection plane :

$$\frac{\frac{screen^{width}}{2.0}}{\tan(FOV)}$$

ou tan(FOV / 2)
En fonction de la tete de l'affichage !

- Etape 2 : Affichage des murs, sols et plafond

Maintenant que nous avons la distance du joueur avec l'axe de projection, nous pouvons determiner la taille des murs en fonction de tout ce qu'on connait, ici nous utiliserons la distance rayon, la distance avec l'axe de projection et une constante (modifiable) pour rendre les murs plus ou moins grand.

$$h\_wall = \left(\frac{const}{rayon_{dist}}\right) \cdot ProjectionPlane_{dist}$$

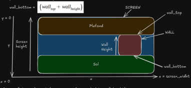
Maintenant que la hauteur du mur est defini, nous devons trouver le point le plus haut du mur et le point le plus bas du mur pourquoi me direz-vous ?
Simplement pour definir sur les ordonnees quels pixels doivent etre des pixel de mur, sol, ou plafond :
y > px_top -> px_plafond
y < px_bottom -> px_sol
y < px_top && y > px_bottom -> px_mur

pour trouver le point le plus haut du mur :

$$wall\_top = \frac{\left(screen_{height} - wall_{height}\right)}{2}$$

pour trouver le point le plus bas du mur :

$$wall\_bottom = \left(wall_{top} + wall_{height}\right)$$



Pour wall_top, cela veut dire que apres (screen_heiht - wall_height) il nous reste l'espace libre sur l'ecran, en faisant / 2, cela nous donne l'espace au dessus du top et en dessous du bottom.
Comme y = 0 quand nous somme au sommet de l'ecran (inverser mathematiquement) le resultat = wall_top

donc logiquement wall_bottom = wall_top + wall_height.

LANCONS MAINTENANT L'AFFICHAGE : JE RAPPEL :

y > px_top -> px_plafond
y < px_bottom -> px_sol
y < px_top && y > px_bottom -> px_mur

donc en fonction de la valeur de y dans une boucle qui commence a 0, on imprimera soit un px de plafond, de sol, ou de mur, en suivant cette logique, il est possible de faire cela avec n'importe quel textures representant un sol, mur ou plafond.

PASSONS AU RENDU DES TEXTURES !!!

6. Rendu des Textures :

Pour ajouter du réalisme, des textures peuvent être appliquées aux murs.

A ce moment, pour chaques colonnes (x) nous allons devoir afficher les textures associes.
Neanmoins, les textures ont une taille fixe, (50px, 50px) ou (100px, 100px) etc...
il faut donc chercher dans la texture quel colonne de pixel afficher pour que ca s'adapte a n'importe quel taille de mur (en longueur comme en hauteur) et que l'affichage reste coherent.
(dans le contexte de cube3d, le sol et le plafond sont des couleurs, donc il faut simplement afficher la couleur correspondante pour le sol ou le plafond sur la scene.)

SINON il faut recuperer l'image de la structure et trouver les coordonnee (x, y) du pixel de la texture a afficher.

IL NOUS FAUT : le pixel precis a afficher de la texture (x, y) que nous nommerons (tex_x, tex_y).
Les Donnees du pixel a afficher un char * nommer pixel_addr.

RECHERCHE DE TEX_X ET TEX_Y :

SOUVENEZ-VOUS nous avons le ratio de wall_hit qui nous permet de savoir ou nous nous situons en dans la face du mur toucher en x (abscisse) avec un ratio entre [0.0, 1.0].
pour trouver le pixel de la texture a avoir en x,
il nous suffit alors de faire ratio * largeur de la texture.

$$tex_x = \left(RATIO \cdot tex_{widht}\right)$$

Pour tex_y, IL FAUT calculer son ratio car wall_git permet de se situe en abscisse.
il nous faut donc un calcul pour calculer le ratio en oordonnee du pixel.

$$RATIO = \frac{\left(y - Wall_{top}\right)}{wall_{height}}$$

y = px que l on essaye d'afficher dans le screen en ordonnee (axe y).
wall_height = hauteur du mur projeter (voir calcul Affichage des murs sols et plafond)

PUIS nous pouvons retrouver le pixel de la texture en Y :

$$tex_y = \left(RATIO \cdot tex_{height}\right)$$

MAINTENANT que tex_x et tex_y est recuperer, l'affichage de la texture va pouvoir se faire de facon coeherente !

PS (JE NE VAIS PAS DETAILLER ICI COMMENT RECUPERER LE PIXEL DANS L'IMAGE DE LA TEXTURE JE VOUS LAISSE CHERCHER COMMENT CELA FONCTIONNE, MAIS TEX_X et TEX_Y EST CE QU'IL VOUS FAUT !)

7. CONCLUSION :

Le raycasting est une technique efficace pour créer des environnements 3D à partir de cartes 2D, et elle a été largement utilisée dans les premiers jeux FPS comme "Wolfenstein 3D". Bien que des techniques plus avancées comme le ray tracing existent aujourd'hui, le raycasting reste une méthode importante dans l'histoire des jeux vidéo.

J'espere que ce tutoriel de raycasting sera utile pour quelqu'un, ce fut un plaisir de le rediger !
(desole pour les fautes j'ai pas eu la fois de corriger mon ortographe xd)
(Je tiens aussi a souligner que des erreurs ont plus ce glisser, ne prenez pas tout au pied de la lettre faite vous votre propre avis !)
ENJOY !